

GrapheinAI

Plateforme SaaS d'Intelligence Graphique Multimodale d'Entreprise

Documentation Complète — Manuel Officiel

Extraire, valider, analyser, interpréter et synthétiser automatiquement les données de vos graphiques scientifiques, financiers et industriels — en toute confiance, sans hallucination de calcul.

Projet	GrapheinAI — SaaS Enterprise d'Intelligence Graphique
Version du système	5.0.0 Enterprise
Statut	Production / SRE Ready
Auteur du document	Chrisostome Mouahé Kouakou
Date	28 juillet 2026

Sommaire

1. Introduction **3**
2. Présentation générale de GrapheinAI **4**
3. Parcours utilisateur complet **5**
4. Les fonctionnalités **9**
5. Architecture logicielle **12**
6. Les technologies **15**
7. Fonctionnement détaillé de l'IA **18**
8. Cycle complet d'une analyse **20**
9. Cas d'utilisation **21**
10. Les points forts **23**
11. Limites **24**
12. Glossaire **25**
13. FAQ **26**
14. Conclusion **28**

1. Introduction

Ce chapitre pose le contexte métier dans lequel s'inscrit GrapheinAI : le problème qu'il résout, pourquoi ce problème persiste malgré les outils existants, et les conséquences concrètes de son absence de solution.

1.1. Contexte

Dans les entreprises, les administrations et le monde académique, une quantité considérable d'informations circule sous forme de graphiques : rapports financiers trimestriels, tableaux de bord industriels, études scientifiques, présentations commerciales. Ces graphiques sont très souvent figés dans des formats non exploitables directement par une machine : images PNG/JPEG, captures d'écran, planches issues de PDF scannés.

1.2. Le problème métier

Lorsqu'une personne a besoin des données chiffrées qui se cachent derrière un graphique — pour les recalculer, les comparer, les injecter dans un autre outil — elle doit aujourd'hui les relire visuellement et les ressaisir manuellement, point par point, dans un tableur. Ce travail est lent, répétitif et sujet à l'erreur humaine, en particulier lorsque le graphique contient de nombreuses séries de données ou des échelles complexes.

1.3. Pourquoi ce problème existe

- De nombreux documents n'existent qu'en image (rapports PDF anciens, captures d'écran d'outils de Business Intelligence sans fonction d'export, photographies de tableaux ou d'écrans).
- Un OCR (reconnaissance optique de caractères) classique sait lire du texte, mais ne comprend pas la structure d'un graphique : il ne sait pas relier une barre à sa valeur sur l'axe, ni une courbe à sa légende.
- Comprendre un graphique demande un raisonnement visuel ET sémantique combiné — une tâche qui dépasse la portée d'un simple outil d'extraction de texte.

1.4. Les conséquences

L'absence d'une solution fiable pour cette tâche entraîne des décisions retardées, des erreurs de calcul lors des ressaisies manuelles, une perte de traçabilité (on ne sait plus comment un chiffre a été obtenu), et une forte dépendance à des analystes humains pour une tâche répétitive à faible valeur ajoutée.

1.5. Pourquoi GrapheinAI a été créé

GrapheinAI a été conçu pour automatiser cette chaîne de bout en bout — extraction, validation, analyse, interprétation — tout en réglant le principal point faible des IA génératives sur ce type de tâche : leur tendance à halluciner des chiffres. Le projet combine une vision par ordinateur déterministe (OpenCV) pour repérer la géométrie du graphique, un modèle de vision-langage (Gemini Flash Vision) pour l'interpréter sémantiquement, un évaluateur mathématique strictement déterministe (SafeCalculator, basé sur l'analyse d'arbre syntaxique Python) pour garantir l'exactitude des calculs, et une étape de validation humaine (Human-in-the-Loop) avant toute analyse.

> **Résumé du chapitre**
>
> GrapheinAI répond à un besoin réel : transformer des graphiques figés en données exploitables et fiables, sans ressaisie manuelle ni erreur de calcul.
>
> Le problème existe parce que l'OCR classique ne comprend pas la structure visuelle d'un graphique.
>
> La réponse de GrapheinAI combine vision par ordinateur, IA générative multimodale et calcul déterministe.

2. Présentation générale de GrapheinAI

Ce chapitre donne une vue d'ensemble de l'application : à qui elle s'adresse, ce qu'elle permet de faire, et les bénéfices concrets qu'elle apporte.

2.1. À quoi sert l'application

GrapheinAI est une plateforme SaaS (Software as a Service) qui permet d'extraire, de valider, d'analyser, d'interpréter et de synthétiser automatiquement les données issues de graphiques scientifiques, financiers, industriels et décisionnels : histogrammes, courbes, camemberts, nuages de points, ainsi que des planches contenant plusieurs graphiques à la fois.

2.2. Qui sont les utilisateurs

D'après les cas d'utilisation documentés (voir chapitre 9), GrapheinAI s'adresse à des profils variés qui ont besoin d'exploiter des données graphiques sans nécessairement maîtriser le développement, l'intelligence artificielle ou la data science : analystes financiers, chercheurs, étudiants, responsables commerciaux et data analysts.

2.3. Les bénéfices apportés

- **Gain de temps :** l'extraction qui prenait des minutes de ressaisie manuelle devient quasi instantanée.
- **Fiabilité des calculs :** le moteur SafeCalculator élimine toute hallucination numérique en n'exécutant que des opérations mathématiques vérifiées.
- **Travail collaboratif :** les espaces de travail (workspaces) permettent de partager une analyse avec une équipe.
- **Exports professionnels :** rapports PDF vectoriels et fichiers Excel prêts à être diffusés.
- **Sobriété numérique :** routage intelligent des requêtes pour limiter les appels aux IA cloud coûteuses en énergie.



Image 1 : *Vue d'ensemble de la page d'accueil de GrapheinAI, point d'entrée de la plateforme.*

3. Parcours utilisateur complet

Ce chapitre décrit, étape par étape, le chemin complet d'un utilisateur dans GrapheinAI : de la création de son compte jusqu'à la gestion de son profil, en passant par l'ensemble du cycle d'analyse d'un graphique. Pour chaque étape, quatre angles sont détaillés : ce que voit l'utilisateur, ce qu'il fait, ce qui se passe côté application, et les technologies mobilisées.

Étape 1 — Création de compte

Ce que voit l'utilisateur. Un formulaire d'inscription (email, mot de passe) sur l'interface Web SPA ou l'application Streamlit.

Ce qu'il fait. Renseigne ses identifiants et valide la création de son compte.

Ce que fait l'application (en coulisses). Le compte est créé via Supabase Auth, qui gère l'identité de l'utilisateur et génère un jeton d'authentification (JWT) pour les sessions futures.

Technologies mobilisées. Supabase Auth, Supabase Client (supabase-py).

Étape 2 — Connexion

Ce que voit l'utilisateur. Un formulaire de connexion (email, mot de passe).

Ce qu'il fait. Saisit ses identifiants et se connecte.

Ce que fait l'application (en coulisses). La requête traverse la chaîne de sécurité : middleware CORS, en-têtes de sécurité OWASP, PromptInjectionGuard, puis le garde Supabase Auth & RLS qui valide le jeton JWT avant d'autoriser l'accès à l'API.

Technologies mobilisées. Supabase Auth, JWT, RLS Guard, middleware CORS.

Étape 3 — Tableau de bord

Ce que voit l'utilisateur. Un tableau de bord listant les sessions d'analyse de l'utilisateur et, le cas échéant, les espaces de travail (workspaces) auxquels il appartient.

Ce qu'il fait. Consulte ses analyses passées ou démarre une nouvelle analyse.

Ce que fait l'application (en coulisses). L'API interroge la table analysis_sessions de la base Supabase PostgreSQL ; les politiques RLS filtrent automatiquement les résultats pour ne montrer que les sessions de l'utilisateur ou de son espace de travail.

Technologies mobilisées. FastAPI, AnalysisSessionManager, Supabase PostgreSQL + RLS.

Étape 4 — Import d'un graphique

Ce que voit l'utilisateur. Une zone de téléversement (glisser-déposer ou bouton) acceptant les formats PNG, JPEG, WEBP et PDF.

Ce qu'il fait. Sélectionne et téléverse l'image ou le PDF contenant le graphique à analyser.

Ce que fait l'application (en coulisses). L'appel POST /api/v1/extract transmet le fichier au PromptInjectionGuard, qui valide sa taille (moins de 20 Mo) et vérifie ses « magic bytes » pour s'assurer qu'il s'agit bien d'un fichier image/PDF légitime et non d'un fichier malveillant déguisé.

Technologies mobilisées. FastAPI, PromptInjectionGuard, Pillow.

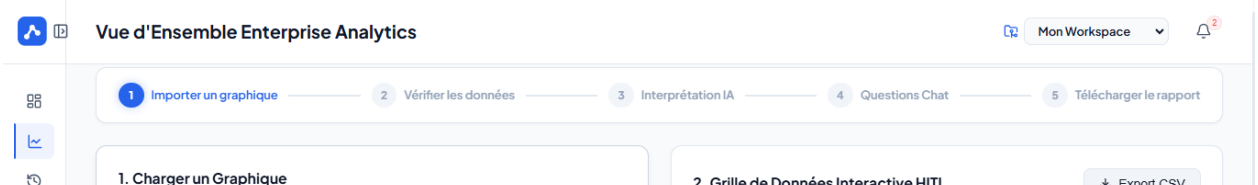


Image 2 : L'utilisateur importe le graphique qui servira de point de départ à toute l'analyse réalisée par GrapheinAI.

Étape 5 — Prévisualisation

Ce que voit l'utilisateur. Un aperçu de l'image importée, telle qu'elle sera analysée.

Ce qu'il fait. Vérifie visuellement que le bon fichier a bien été chargé avant de lancer l'extraction.

Ce que fait l'application (en coulisses). L'image est normalisée par Pillow (redimensionnement, conversion de format) avant d'être transmise au pipeline d'extraction.

Technologies mobilisées. Pillow (PIL).

Étape 6 — Extraction des données

Ce que voit l'utilisateur. Un indicateur de traitement, puis une grille de données pré-remplie avec les valeurs détectées dans le graphique.

Ce qu'il fait. Attend la fin du traitement automatique (aucune action requise).

Ce que fait l'application (en coulisses). Le PipelineAgent orchestre la chaîne d'extraction : l'OCREngine (OpenCV) détecte d'abord les boîtes englobantes du texte, des axes et des légendes ; le ReasoningAgent (Gemini Flash Vision) analyse ensuite l'image, guidé par ces boîtes, pour produire une extraction structurée (schéma Pydantic ChartExtraction) ; enfin le ChartIntelligenceEngine réconcilie la géométrie détectée par OpenCV avec la lecture sémantique de Gemini.

Technologies mobilisées. OpenCV, Gemini Flash Vision, Pydantic, ChartIntelligenceEngine.

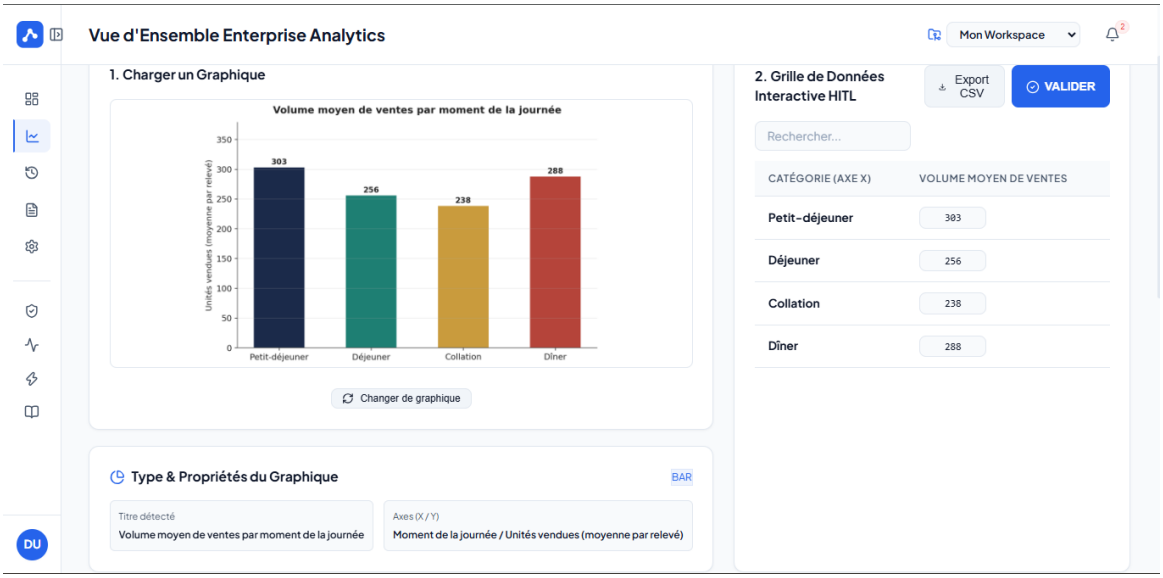


Image 3 : *En quelques secondes, GrapheinAI transforme l'image du graphique en données structurées et modifiables.*

Étape 7 — Validation des données (Human-in-the-Loop)

Ce que voit l'utilisateur. Une grille de données interactive affichant chaque valeur extraite, accompagnée d'un score de confiance (VERY_HIGH, HIGH, MEDIUM ou LOW).

Ce qu'il fait. Corrige, si nécessaire, les valeurs mal détectées, puis valide la grille.

Ce que fait l'application (en coulisses). Le ValidationAgent calcule le score de confiance en croisant trois signaux : l'alignement OCR, le chevauchement géométrique entre les zones détectées et la plausibilité numérique des valeurs.

Technologies mobilisées. ValidationAgent ; Streamlit-AgGrid dans la version Streamlit.

Étape 8 — Analyse IA / question-réponse

Ce que voit l'utilisateur. Une zone de question en langage naturel, où l'utilisateur peut interroger ses données.

Ce qu'il fait. Pose une question libre, par exemple : « Quelle est la croissance entre janvier et mars ? »

Ce que fait l'application (en coulisses). L'appel POST /api/v1/ask passe par le PromptInjectionGuard, puis le QuestionIntentClassifier catégorise la question (CALCULATION, STATISTICS, DATA_POINT, TREND, COMPARISON ou GENERAL_ANALYSIS) tandis que le ClassifierAgent (XGBoost) évalue sa complexité pour la router efficacement. Le RetrievalAgent interroge ensuite l'index FAISS pour retrouver des exemples de questions-réponses similaires. Si la question est un calcul, le ReasoningAgent formule l'expression mathématique correspondante et le SafeCalculator l'évalue de façon déterministe ; sinon, le ReasoningAgent raisonne directement à partir du contexte.

Technologies mobilisées. FastAPI, QuestionIntentClassifier, XGBoost, FAISS, Gemini, SafeCalculator (AST).

Étape 9 — Conversation avec l'assistant

Ce que voit l'utilisateur. Un fil de discussion affichant les échanges successifs avec l'assistant.

Ce qu'il fait. Enchaîne plusieurs questions en gardant le contexte de la conversation.

Ce que fait l'application (en coulisses). Le ConversationManager conserve la mémoire des échanges précédents pour que chaque nouvelle réponse tienne compte du fil de la conversation.

Technologies mobilisées. ConversationManager.

Étape 10 — Génération du rapport PDF

Ce que voit l'utilisateur. Un bouton d'export permettant de générer un rapport officiel.

Ce qu'il fait. Clique pour générer et télécharger le rapport.

Ce que fait l'application (en coulisses). L'appel POST /api/v1/export/pdf déclenche le PdfReportGenerator (ReportLab), qui assemble en un document vectoriel multi-pages les graphiques, les tableaux de données, les indicateurs d'explicabilité (XAI) et les recommandations métier.

Technologies mobilisées. ReportLab.

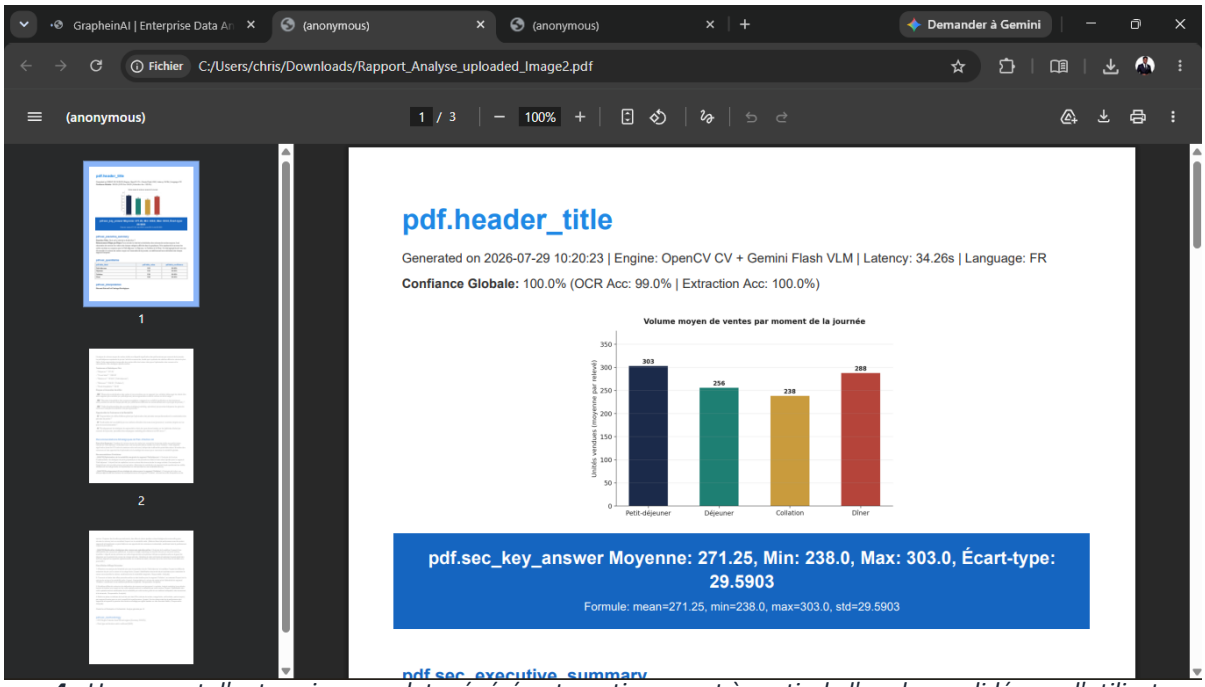


Image 4 : *Un rapport d'entreprise complet, généré automatiquement à partir de l'analyse validée par l'utilisateur.*

Étape 11 — Historique

Ce que voit l'utilisateur. Une liste chronologique des sessions d'analyse passées.

Ce qu'il fait. Consulte ou rouvre une session antérieure.

Ce que fait l'application (en coulisses). L'API lit la table analysis_sessions, dont l'accès est strictement isolé par les politiques RLS de Supabase (chaque utilisateur ne voit que ses propres sessions, ou celles de son espace de travail).

Technologies mobilisées. Supabase PostgreSQL + RLS.

Étape 12 — Partage

Ce que voit l'utilisateur. Une option d'invitation à un espace de travail (workspace), avec le choix d'un rôle.

Ce qu'il fait. Invite un collègue et lui attribue un rôle : administrateur, analyste ou lecteur (admin, analyst, viewer).

Ce que fait l'application (en coulisses). Une nouvelle ligne est ajoutée dans la table workspace_members, un email d'invitation est envoyé via le service Resend, et les droits d'accès RBAC correspondants sont appliqués.

Technologies mobilisées. Supabase, Resend SDK, RBAC.

Étape 13 — Gestion du profil

Ce que voit l'utilisateur. Une page de profil (informations personnelles, langue préférée, réinitialisation du mot de passe).

Ce qu'il fait. Met à jour ses informations ou réinitialise son mot de passe.

Ce que fait l'application (en coulisses). L'EmailService (Resend) envoie l'email de réinitialisation de mot de passe ; le LanguageManager adapte automatiquement la langue de l'interface et des réponses de l'IA (français ou anglais).

Technologies mobilisées. Resend SDK, LanguageManager (langdetect).

Conseil

Le principe Human-in-the-Loop (étape 7) est le pilier de confiance de GrapheinAI : aucune donnée extraite par l'IA n'est utilisée pour un calcul ou une analyse tant que l'utilisateur ne l'a pas explicitement validée.

Résumé du chapitre

Le parcours utilisateur suit un fil logique : compte → connexion → import → extraction → validation humaine → analyse conversationnelle → export → historique/partage.

Chaque étape mobilise des technologies précises, de la vision par ordinateur à la génération de PDF vectoriel.

La validation humaine (HITL) intervient systématiquement avant toute analyse ou calcul, garantissant la fiabilité du résultat final.

4. Les fonctionnalités

Ce chapitre détaille les sept piliers fonctionnels de GrapheinAI, tels que présentés dans sa documentation technique. Chaque section explique l'objectif de la fonctionnalité, son fonctionnement, son intérêt pour l'utilisateur, un scénario réel d'usage, et les technologies qui la rendent possible.

4.1. 🔍 Extraction Hybride Vision + OCR

Objectif. Transformer l'image d'un graphique en données structurées et exploitables.

Fonctionnement. Deux couches d'analyse se combinent : une segmentation géométrique bas niveau avec OpenCV, qui détecte les contours, les zones de texte, les axes et les graduations sous forme de boîtes englobantes [x, y, w, h] ; et une analyse sémantique par modèle de vision-langage (Gemini Flash Vision), qui comprend la structure visuelle globale et extrait les séries de données dans un schéma Pydantic strict (ChartExtraction, ExtractedDataPoint).

Intérêt. Cette double approche est plus robuste qu'un OCR seul (qui ne comprend pas la structure d'un graphique) ou qu'un modèle de vision seul (qui peut se tromper sur des repères géométriques précis).

Scénario réel. Un analyste reçoit une capture d'écran d'un tableau de bord industriel sans accès aux données sources ; GrapheinAI en extrait directement les séries chiffrées.

Technologies utilisées. OpenCV (opencv-python-headless), Google Gemini Flash Vision, Pydantic.

Illustration suggérée : capture de la grille de données générée automatiquement après import (voir chapitre 3, étape 6).

4.2. 🎛️ Workflow HITL (Human-in-the-Loop) & Grille d'Édition

Objectif. Permettre à l'utilisateur de corriger et de valider les données avant toute analyse.

Fonctionnement. Une grille de données interactive permet d'inspecter, de corriger et de valider les valeurs extraites. Le ValidationAgent calcule un score de confiance global (VERY_HIGH, HIGH, MEDIUM, LOW) fondé sur l'alignement OCR, le chevauchement géométrique et la plausibilité numérique.

Intérêt. Cette étape place le contrôle final entre les mains de l'utilisateur : l'IA propose, l'humain dispose. C'est un rempart essentiel contre les erreurs d'extraction.

Scénario réel. Un chercheur remarque qu'un point de donnée a été mal lu sur un axe logarithmique ; il le corrige directement dans la grille avant de lancer l'analyse statistique.

Technologies utilisées. ValidationAgent, Streamlit-AgGrid (dans l'interface Streamlit).

Illustration suggérée : capture de la grille de validation avec les scores de confiance affichés par ligne.

4.3. 🧮 Raisonnement Déterministe Anti-Hallucination (SafeCalculator AST)

Objectif. Garantir l'exactitude absolue des calculs effectués sur les données du graphique.

Fonctionnement. Les expressions mathématiques générées par l'IA (par exemple pour calculer un taux de croissance) sont évaluées par un analyseur d'arbre syntaxique abstrait Python (module ast), strictement restreint aux opérations arithmétiques autorisées (+, -, *, /, **) et à un jeu de fonctions statistiques (min, max, sum, avg, abs, round). L'usage d'eval() est proscrit.

Intérêt. Contrairement à un modèle de langage qui peut halluciner un résultat numérique, ce moteur ne peut produire que le résultat mathématiquement exact de l'expression évaluée, sans possibilité d'exécuter du code arbitraire.

Scénario réel. Un responsable commercial demande la variation en pourcentage entre deux valeurs d'un graphique ; le résultat retourné est garanti mathématiquement exact, sans marge d'erreur liée à l'IA générative.

Technologies utilisées. Module Python AST (SafeCalculator).

Illustration suggérée : non applicable — fonctionnement interne, sans écran dédié.

4.4. Routage Intelligent & Recherche Vectorielle (RAG FAISS)

Objectif. Répondre plus vite et de façon plus pertinente en réutilisant des connaissances déjà indexées, tout en limitant les appels aux IA cloud.

Fonctionnement. Le QuestionIntentClassifier catégorise chaque requête (CALCULATION, STATISTICS, DATA_POINT, TREND, COMPARISON, GENERAL_ANALYSIS). Le ClassifierAgent, basé sur XGBoost, évalue ensuite la complexité de la question pour la router vers un traitement rapide local ou vers la chaîne multimodale complète. L'index vectoriel FAISS, alimenté par des embeddings Sentence-Transformers, permet une recherche de contexte et d'exemples Q&R en sous-milliseconde.

Intérêt. Ce routage évite de solliciter systématiquement un modèle d'IA générative distant pour des questions simples, ce qui accélère les réponses et réduit les coûts et l'empreinte énergétique.

Scénario réel. Un étudiant pose une question de comparaison déjà rencontrée dans une session précédente ; le RetrievalAgent retrouve un exemple similaire dans l'index FAISS local, sans appel supplémentaire au modèle cloud.

Technologies utilisées. XGBoost, Scikit-Learn, FAISS (faiss-cpu), Sentence-Transformers (all-MiniLM-L6-v2).

Illustration suggérée : non applicable — fonctionnement interne, sans écran dédié.

4.5. Synthèse Scientifique Narrative & Recommandations Métier

Objectif. Transformer des données brutes en une interprétation claire et en recommandations actionnables.

Fonctionnement. Le GraphInterpreter génère automatiquement une synthèse narrative mettant en avant les tendances globales, les pics, les minima et les corrélations. Le RecommendationEngine formule ensuite des recommandations stratégiques adaptées au secteur (Finance, Santé, Immobilier, Industrie, Tech) et au niveau d'expertise de l'utilisateur. L'ExplainabilityEngine (XAI) documente dix indicateurs de traçabilité du raisonnement (données utilisées, formules AST évaluées, extraits RAG mobilisés, temps d'exécution, version du modèle).

Intérêt. L'utilisateur ne reçoit pas seulement des chiffres, mais une lecture experte et contextualisée, avec la preuve de la façon dont elle a été obtenue.

Scénario réel. Un responsable Finance obtient, en plus des chiffres extraits, un résumé exécutif et des actions recommandées adaptées à son secteur.

Technologies utilisées. GraphInterpreter, InsightAgent, RecommendationEngine, ExplainabilityEngine.

Illustration suggérée : capture du rapport de synthèse narrative et des recommandations affichées à l'écran.

4.6. Exports Officiels PDF & Excel High-Definition

Objectif. Produire des livrables professionnels prêts à être partagés ou archivés.

Fonctionnement. Le moteur PDF vectoriel (pdf_generator.py, basé sur ReportLab) génère instantanément des rapports PDF d'entreprise multi-pages incluant graphiques, tableaux de données, indicateurs XAI et recommandations. L'export Excel, via openpyxl et xlsxwriter, produit un classeur structuré multi-feuilles.

Intérêt. Ces exports permettent d'intégrer directement le résultat d'une analyse GrapheinAI dans un rapport d'entreprise, une soutenance ou un dossier de présentation, sans reformatage manuel.

Scénario réel. Un data analyst exporte son analyse validée en PDF pour l'inclure telle quelle dans un comité de direction.

Technologies utilisées. ReportLab, OpenPyXL, XlsxWriter.

Illustration suggérée : capture de l'aperçu ou du téléchargement du rapport PDF généré (voir chapitre 3, étape 10).

4.7. Authentification Multi-Tenant & Sécurité SRE

Objectif. Garantir la confidentialité des données de chaque client et la fiabilité opérationnelle de la plateforme.

Fonctionnement. Supabase Auth gère l'authentification par jetons JWT ; les espaces de travail partagés (workspaces) et les rôles RBAC (Admin, Analyst, Viewer) structurent les droits d'accès, appliqués strictement par les politiques RLS de PostgreSQL. Le PromptInjectionGuard filtre les tentatives de jailbreak, nettoie le HTML/scripts et valide les en-têtes « magic bytes » des fichiers téléversés. Des endpoints /health, /status, /version et /metrics assurent l'observabilité, avec rotation automatique des logs (Loguru) et sauvegardes programmées.

Intérêt. Cette architecture rend GrapheinAI directement utilisable en environnement d'entreprise multi-clients, avec une isolation stricte des données de chaque espace de travail.

Scénario réel. Une entreprise cliente crée un espace de travail dédié à son équipe finance, invisible et inaccessible aux autres clients de la plateforme, garanti au niveau même de la base de données.

Technologies utilisées. Supabase Auth & RLS, PromptInjectionGuard, Loguru, scripts/backup_manager.py.

Illustration suggérée : non applicable — fonctionnement interne, sans écran dédié.

Résumé du chapitre

Les sept fonctionnalités de GrapheinAI couvrent tout le cycle : extraction fiable, contrôle humain, calcul déterministe, routage intelligent, interprétation métier, export professionnel et sécurité multi-tenant.

Le fil conducteur commun est la fiabilité : chaque fonctionnalité ajoute une garantie supplémentaire contre l'erreur ou l'hallucination.

5. Architecture logicielle

Ce chapitre décrit l'architecture technique de GrapheinAI : son style architectural général, ses couches (frontend, backend, base de données, IA, sécurité) et la manière dont ces composants interagissent.

5.1. Style architectural

GrapheinAI repose sur un motif Hexagonal (Ports & Adapters), qui sépare rigoureusement trois couches :

- **La couche d'interface (adaptateurs entrants) :** l'interface Web SPA légère (HTML5/Vanilla CSS), l'application Streamlit interactive, et l'API REST FastAPI (ASGI).
- **Le cœur métier (Domain Core) :** l'orchestrateur PipelineAgent, les agents d'IA spécialisés, l'analyseur AST déterministe, les moteurs de calculs statistiques et les générateurs de rapports.
- **La couche d'infrastructure (adaptateurs sortants) :** la persistance PostgreSQL Supabase (avec Row Level Security), le moteur de recherche vectoriel FAISS local, la génération de documents PDF (ReportLab) et les services d'emails transactionnels (Resend).

Cette séparation permet de faire évoluer chaque couche indépendamment : par exemple, remplacer le frontend SPA par une autre interface sans toucher au cœur métier.

5.2. Frontend

Deux interfaces clientes sont proposées : une Single Page Application en HTML5 et CSS natif (sans framework JavaScript lourd, pour un bundle inférieur à 200 Ko), et une application Streamlit alternative, utilisable localement, sur Streamlit Community Cloud ou sur Hugging Face Spaces. L'interface Streamlit s'appuie sur Streamlit-AgGrid pour la grille d'édition HITL, et sur Plotly/Altair pour des visualisations interactives.

5.3. Backend

Le backend est une API REST développée avec FastAPI (0.110.0 ou supérieur), exécutée sur le serveur ASGI Uvicorn pour une gestion asynchrone haute performance. Pydantic (2.6.0 ou supérieur) valide rigoureusement l'ensemble des modèles de données échangés. L'API expose plus de 25 endpoints documentés automatiquement via Swagger/OpenAPI.

5.4. Base de données

La persistance repose sur Supabase PostgreSQL. Le schéma principal (supabase/schema.sql) comprend notamment trois tables clés :

Table	Rôle
workspaces	Espaces de travail collaboratifs, rattachés à un propriétaire (owner_id).
workspace_members	Membres d'un espace de travail, avec un rôle RBAC : admin, analyst ou viewer.
analysis_sessions	Sessions d'analyse : titre et type du graphique, données extraites (JSONB), score de confiance, statut.

5.5. APIs

Les endpoints explicitement documentés dans les fichiers fournis sont POST /api/v1/extract (extraction d'un graphique), POST /api/v1/ask (question-réponse), POST /api/v1/export/pdf (génération du rapport), ainsi que les endpoints de supervision /health, /status, /version et /metrics. La documentation exhaustive des 25+ endpoints est renvoyée vers un fichier API.md, référencé dans le projet mais non fourni pour la rédaction de ce document.

5.6. Authentification

L'authentification repose sur Supabase Auth (jetons JWT). Chaque requête entrante traverse une chaîne de sécurité : middleware CORS, en-têtes de sécurité OWASP, PromptInjectionGuard, puis le garde

Supabase Auth & RLS. Au niveau des espaces de travail, un modèle RBAC distingue trois rôles : administrateur, analyste et lecteur.

5.7. Intelligence artificielle

Le cœur IA est orchestré par le PipelineAgent, qui coordonne dix agents spécialisés (classification d'intention, OCR, raisonnement multimodal, calcul déterministe, recherche vectorielle, validation, interprétation, recommandation et explicabilité). Le détail complet de ce pipeline est présenté au chapitre 7.

5.8. Stockage

Trois mécanismes de stockage coexistent : Supabase PostgreSQL pour les données structurées et persistantes, un cache hybride (LRU en mémoire vive via CacheManager, et DiskCache persistant sur disque) pour éviter de retraiter des images déjà analysées, et des fichiers de logs tournants gérés par Loguru (RotatingFileHandler).

5.9. Sécurité

La sécurité est traitée à plusieurs niveaux : middleware CORS et en-têtes OWASP au niveau réseau, PromptInjectionGuard contre les tentatives de jailbreak et les fichiers malveillants (vérification des « magic bytes »), Row Level Security PostgreSQL pour l'isolation multi-tenant au niveau de la base de données, et le SDK Sentry pour la remontée en temps réel des erreurs de production. Ce sujet est approfondi au chapitre 5 de l'Architecture technique et repris dans la FAQ.

5.10. Interactions entre les composants

Le schéma ci-dessous représente, sous forme textuelle, le flux général d'une requête à travers les différentes couches de GrapheinAI.

Schéma d'architecture (représentation textuelle)

```
Client (SPA HTML5 / React / Streamlit)
   |
   v
CORS -> En-têtes de sécurité OWASP -> PromptInjectionGuard -> Supabase Auth & RLS
   |
   v
FastAPI REST Router -> AnalysisSessionManager -> CacheManager (LRU + DiskCache)
   |
   v
PipelineAgent (orchestrateur central)
   |-- QuestionIntentClassifier
   |-- OCREngine (OpenCV)
   |-- ChartIntelligenceEngine
   |-- ReasoningAgent (Gemini Flash Vision)
   |-- SafeCalculator (AST)
   |-- RetrievalAgent (FAISS)
   |-- ValidationAgent
   |-- GraphInterpreter
   |-- RecommendationEngine
   `-- ExplainabilityEngine (XAI)
   |
   v
Supabase PostgreSQL (RLS) | Logs (Loguru) | PDFReportGenerator (ReportLab) | EmailService
(Resend)
```

Résumé du chapitre

GrapheinAI suit une architecture hexagonale à trois couches : interface, cœur métier IA, infrastructure.

Le backend FastAPI orchestre un pipeline de dix agents IA spécialisés autour d'une base Supabase PostgreSQL sécurisée par RLS.

> La sécurité et l'observabilité (CORS, PromptInjectionGuard, Sentry, endpoints de santé) sont intégrées nativement, pas ajoutées après coup.

6. Les technologies

Ce chapitre présente les technologies utilisées par GrapheinAI. Sept technologies « piliers » — celles qui portent la valeur différenciante du projet — sont détaillées sous forme de fiches pédagogiques (définition, rôle, choix, avantages, exemple). Les autres technologies, tout aussi réelles mais plus classiques, sont regroupées dans un tableau de synthèse.

6.1. Technologies piliers

Google Gemini Flash Vision (VLM)

Définition. Un modèle de vision-langage (Vision-Language Model) : une IA générative capable de traiter simultanément une image et du texte, et de raisonner sur les deux à la fois.

Rôle dans GrapheinAI. C'est le cœur du ReasoningAgent : il analyse l'image du graphique, guidé par les boîtes détectées par OpenCV, extrait les séries de données et génère les raisonnements contextualisés lors des questions-réponses.

Pourquoi ce choix. Sa fenêtre de contexte de plus d'1 million de tokens et son traitement multimodal natif permettent une inférence sous la seconde, essentielle pour une expérience utilisateur fluide.

Avantages. Compréhension sémantique fine (par exemple distinguer une légende d'un titre), raisonnement contextualisé, réponses bilingues.

Exemple concret. Face à un histogramme, Gemini identifie que la troisième barre correspond au mois de mars et que sa valeur avoisine 85, en s'appuyant sur la boîte englobante fournie par OpenCV autour de cette barre.

OpenCV (Computer Vision / OCR géométrique)

Définition. Une bibliothèque C++ de traitement d'image haute vitesse, très largement utilisée en vision par ordinateur.

Rôle dans GrapheinAI. Il assure la détection des contours, le seuillage adaptatif et l'extraction des boîtes englobantes [x, y, w, h] autour des textes, axes et légendes d'un graphique (OCREngine).

Pourquoi ce choix. Sa rapidité et sa robustesse géométrique en font un complément indispensable au modèle de vision-langage, qui seul peut se tromper sur des repères précis (échelle, position exacte d'un point).

Avantages. Rapidité d'exécution, indépendance vis-à-vis d'un service cloud pour cette étape, précision géométrique.

Exemple concret. Avant d'interroger Gemini, OpenCV délimite précisément la zone de la légende du graphique, ce qui guide l'attention du modèle de vision-langage vers le bon endroit de l'image.

SafeCalculator — analyseur d'arbre syntaxique Python (module ast)

Définition. Un évaluateur mathématique qui ne lit pas une expression comme du texte libre, mais qui la découpe d'abord en une structure logique (l'arbre syntaxique), puis ne calcule que ce qui est explicitement autorisé.

Rôle dans GrapheinAI. Il évalue toutes les expressions mathématiques générées par l'IA — croissances, ratios, sommes — de façon totalement déterministe, sans jamais exécuter de code arbitraire.

Pourquoi ce choix. C'est la pièce maîtresse de la promesse « zéro hallucination de calcul » de GrapheinAI : un LLM peut se tromper en écrivant un résultat, mais il ne peut pas tromper un évaluateur AST qui recalcule lui-même la vraie valeur.

Avantages. Exactitude mathématique garantie à 100 %, sécurité absolue (aucune exécution de code arbitraire, contrairement à un eval() classique), auditabilité de chaque calcul.

Exemple concret. Pour la question « quelle est la variation entre 40 et 85 ? », Gemini propose l'expression (85 - 40) / 40 * 100, et SafeCalculator la recalcule lui-même pour retourner 112.5, sans jamais faire confiance au chiffre que l'IA aurait pu écrire directement.

FAISS + Sentence-Transformers (RAG local)

Définition. FAISS (Facebook AI Similarity Search) est une base de données vectorielle locale, optimisée pour retrouver très rapidement les éléments les plus proches d'une requête. Sentence-Transformers convertit du texte en vecteurs numériques denses (embeddings) qui capturent son sens.

Rôle dans GrapheinAI. Le RetrievalAgent s'en sert pour retrouver, en sous-milliseconde, des exemples de questions-réponses ou du contexte sectoriel pertinents, afin d'enrichir le raisonnement du modèle de vision-langage (Retrieval-Augmented Generation).

Pourquoi ce choix. Fonctionner en local, sans appel réseau, rend cette recherche quasi instantanée et gratuite en ressources par rapport à un appel systématique à un LLM cloud.

Avantages. Rapidité, réduction du nombre d'appels aux IA génératives distantes, donc réduction des coûts et de l'empreinte énergétique.

Exemple concret. Lorsqu'un utilisateur pose une question de comparaison déjà rencontrée par le passé, FAISS retrouve l'exemple similaire dans son index local, sans re-solliciter Gemini pour reformuler le raisonnement depuis zéro.

XGBoost (ClassifierAgent)

Définition. Un algorithme de Gradient Boosting, une famille de modèles d'apprentissage automatique supervisé réputée pour sa précision sur des données structurées.

Rôle dans GrapheinAI. Il détermine la complexité d'une question (SIMPLE ou COMPLEXE) afin de la router soit vers un traitement rapide local, soit vers la chaîne multimodale complète impliquant Gemini.

Pourquoi ce choix. Ce classifieur léger évite de mobiliser systématiquement le modèle de vision-langage, le plus coûteux du pipeline, pour des questions qui peuvent être traitées plus simplement.

Avantages. Rapidité d'inférence, faible coût de calcul, bonne précision sur ce type de classification.

Exemple concret. Une question du type « quelle est la valeur en mars ? » est classée SIMPLE par le ClassifierAgent et traitée par un chemin rapide, tandis qu'une demande d'analyse comparative multi-graphiques est classée COMPLEXE.

Supabase (Auth + PostgreSQL + Row Level Security)

Définition. Une plateforme BaaS (Backend as a Service) qui fournit une base de données PostgreSQL, un système d'authentification et un mécanisme de sécurité au niveau des lignes (Row Level Security).

Rôle dans GrapheinAI. Elle gère les identités (JWT), la persistance des sessions d'analyse, les espaces de travail collaboratifs et l'isolation stricte des données entre clients (multi-tenant).

Pourquoi ce choix. La RLS déplace la logique de sécurité au niveau de la base de données elle-même : même une erreur dans le code applicatif ne peut pas faire fuiter les données d'un autre client, puisque la base les filtre elle-même.

Avantages. Sécurité renforcée « by design », gain de temps de développement (authentification et base de données prêtes à l'emploi), écosystème PostgreSQL éprouvé.

Exemple concret. La politique RLS de la table analysis_sessions n'autorise un utilisateur à lire une session que si auth.uid() correspond au propriétaire, ou si la session appartient à un espace de travail dont il est membre.

ReportLab (génération de PDF vectoriel)

Définition. Un moteur Python de génération de documents PDF, capable de composer des mises en page complexes (graphiques, tableaux, texte) sous forme de fichiers vectoriels.

Rôle dans GrapheinAI. Il alimente le module pdf_generator.py, qui produit les rapports d'analyse officiels multi-pages : graphiques, tableaux de données, métriques XAI et recommandations.

Pourquoi ce choix. Un PDF vectoriel reste net à n'importe quel niveau de zoom et s'imprime avec une qualité professionnelle, contrairement à une simple capture d'écran.

Avantages. Mise en page précise et reproductible, fichiers légers, rendu professionnel adapté à une diffusion en entreprise.

Exemple concret. En un clic, un utilisateur obtient un rapport PDF multi-pages prêt à être joint à un email ou présenté en comité de direction.

6.2. Autres technologies du projet

Le tableau suivant recense les autres technologies documentées dans GrapheinAI, plus classiques mais tout aussi nécessaires au bon fonctionnement de la plateforme.

Technologie	Rôle dans GrapheinAI
FastAPI	Framework Python ASGI qui expose l'API REST de production (25+ endpoints), avec documentation OpenAPI/Swagger automatique.
Uvicorn	Serveur ASGI ultra-rapide qui exécute l'application FastAPI en production avec une haute concurrence.
Pydantic	Validation stricte de tous les modèles de données échangés (graphiques, séries, profils utilisateurs, réponses XAI).
Pillow (PIL)	Chargement, redimensionnement, conversion de formats et découpage des images téléversées.
NumPy & Pandas	Calculs statistiques exacts (moyennes, médianes, écarts-types) et détection d'anomalies (Z-score, IQR).
OpenPyXL & XlsxWriter	Génération des fichiers Excel exportables, avec formules et mise en forme.
Streamlit	Interface utilisateur alternative, déployable sur Streamlit Community Cloud ou Hugging Face Spaces.
Streamlit-AgGrid	Grille d'édition interactive utilisée pour le workflow HITL dans l'interface Streamlit.
Plotly & Altair	Visualisations interactives côté frontend dans l'application Streamlit.
Resend SDK	Envoi des emails transactionnels : réinitialisation de mot de passe, invitations, alertes d'administration.
Loguru	Logging structuré avec rotation et rétention automatique des fichiers de logs.
DiskCache & Tenacity	Cache persistant des extractions VLM et gestion des tentatives de réessai (retry/backoff) sur les appels réseau.
PromptInjectionGuard	Règles de filtrage (regex) et vérification des « magic bytes » contre les tentatives de jailbreak et les fichiers malveillants.
LanguageManager (langdetect)	Détection automatique de la langue et traduction de l'interface et des réponses (français / anglais).
Pytest, Ruff, Black	Suite de tests automatisés (28 fichiers, 180+ assertions), linting et formatage strict du code Python.
Sentry SDK	Suivi des erreurs et monitoring de performance applicative en production.

Résumé du chapitre

Sept technologies « piliers » portent la promesse différenciante de GrapheinAI, en particulier l'évaluateur AST qui élimine les hallucinations de calcul.

Une quinzaine de technologies complémentaires assurent le socle applicatif : API, validation, cache, emails, tests, monitoring.

Chaque choix technologique répond à un besoin précis documenté dans le projet, sans surcouche superflue.

7. Fonctionnement détaillé de l'IA

Ce chapitre déroule, étape par étape, le pipeline d'intelligence artificielle de GrapheinAI, depuis l'import du graphique jusqu'à la génération du rapport final. Chaque étape technique est accompagnée d'une analogie simple pour la rendre accessible à un lecteur non technique.

1. Import du graphique

L'image ou le PDF téléversé par l'utilisateur est réceptionné, validé par le PromptInjectionGuard (taille, format, magic bytes) puis normalisé par Pillow.

Analogie. C'est comme confier un document papier à un assistant : avant de le lire, on vérifie d'abord qu'il s'agit bien du bon type de document et qu'il n'est pas endommagé.

2. Traitement de l'image (OCR géométrique OpenCV)

L'OCREngine analyse l'image pour repérer précisément où se trouvent les axes, les graduations, les légendes et les zones de texte, sous forme de boîtes englobantes [x, y, w, h].

Analogie. C'est comme un architecte qui trace d'abord les contours d'un plan avant d'y ajouter les détails : on cartographie la structure avant de l'interpréter.

3. Extraction des données (Gemini Flash Vision)

Guidé par les boîtes détectées à l'étape précédente, le ReasoningAgent (Gemini) analyse l'image dans son ensemble et produit une extraction structurée des séries de données, conforme à un schéma Pydantic strict.

Analogie. C'est comme un guide touristique qui suit un plan précis pour ne rien manquer, plutôt que de découvrir un lieu au hasard : la carte géométrique oriente la lecture sémantique de l'IA.

4. Validation

Le ValidationAgent croise les résultats de l'OCR géométrique et de la lecture VLM pour calculer un score de confiance par valeur, puis présente le tout dans une grille modifiable par l'utilisateur (HITL).

Analogie. C'est comme un relecteur qui compare une traduction avec le texte original avant de la valider : il ne fait pas confiance à une seule source.

5. Raisonnement (classification d'intention, routage et RAG)

Le QuestionIntentClassifier catégorise la question posée, le ClassifierAgent (XGBoost) évalue sa complexité pour choisir le bon chemin de traitement, et le RetrievalAgent interroge l'index FAISS pour retrouver des exemples pertinents avant de raisonner.

Analogie. C'est comme un standardiste qui redirige un appel vers le bon service — le comptable pour un calcul, l'expert pour une question complexe — après avoir d'abord consulté un recueil de cas similaires déjà traités.

6. Calculs (SafeCalculator AST)

Toute expression mathématique proposée par l'IA est évaluée par l'analyseur d'arbre syntaxique Python, qui ne peut exécuter que des opérations arithmétiques et statistiques strictement autorisées.

Analogie. C'est comme une calculatrice de poche : elle ne peut faire QUE des opérations précises, elle ne peut pas « inventer » un résultat, contrairement à une IA générative qui produit du texte librement.

7. Génération des réponses (interprétation et recommandations)

Le GraphInterpreter rédige une synthèse narrative des tendances et anomalies, tandis que le RecommendationEngine formule des recommandations adaptées au secteur métier de l'utilisateur.

Analogie. C'est comme un analyste qui, après avoir vérifié ses chiffres, rédige une note claire expliquant ce qu'ils signifient et ce qu'il conviendrait de faire.

8. Génération du rapport (ReportLab et explicabilité XAI)

Le PdfReportGenerator met en forme un rapport PDF vectoriel complet, tandis que l'ExplainabilityEngine y intègre dix indicateurs de traçabilité : données utilisées, formules AST évaluées, extraits RAG mobilisés, temps d'exécution, version du modèle.

Analogie. C'est comme un secrétaire qui met en forme un rapport final professionnel, en y joignant systématiquement toutes les pièces justificatives du raisonnement qui a mené à chaque conclusion.

Pipeline IA — vue textuelle

```
Import -> OCR (OpenCV) -> Extraction (Gemini VLM) -> Validation (HITL)
   -> Intent + Routage (XGBoost) + RAG (FAISS) -> Calcul (SafeCalculator AST)
   -> Interprétation (GraphInterpreter + RecommendationEngine)
   -> Rapport (ReportLab + ExplainabilityEngine XAI)
```

Résumé du chapitre

Le pipeline IA de GrapheinAI enchaîne huit étapes, de l'import du graphique à la génération du rapport final.
Chaque étape combine une technologie déterministe (OpenCV, AST) et une technologie générative (Gemini), pour cumuler la richesse de l'IA et la fiabilité du calcul classique.
L'explicabilité (XAI) n'est pas une fonctionnalité annexe mais une étape à part entière du pipeline.

8. Cycle complet d'une analyse

Ce chapitre rassemble, sous forme d'un unique diagramme, l'ensemble du flux parcouru par une analyse dans GrapheinAI — de la première interaction de l'utilisateur jusqu'à l'obtention du rapport final. Il synthétise les chapitres 3 et 7.

Cycle complet d'une analyse

```
Utilisateur
   |  (téléverse un graphique : PNG / JPEG / WEBP / PDF)
   v
Import  --(PromptInjectionGuard : taille, magic bytes)-->
   v
Prétraitement  (Pillow : normalisation ; OpenCV : boîtes englobantes)
   v
Extraction  (Gemini Flash Vision : ChartExtraction structurée)
   v
Validation  (ValidationAgent : score de confiance + grille HITL)
   v
   [ Pause : correction et confirmation par l'utilisateur ]
   v
Analyse  (QuestionIntentClassifier + ClassifierAgent XGBoost + RetrievalAgent FAISS)
   v
Calcul  (SafeCalculator AST, si la question est un calcul)
   v
Réponse  (GraphInterpreter + RecommendationEngine + ExplainabilityEngine XAI)
   v
Rapport  (PdfReportGenerator ReportLab / Export Excel)
```

Deux points méritent d'être soulignés dans ce cycle. D'abord, la pause de validation humaine (HITL) qui intervient obligatoirement entre l'extraction et l'analyse : aucune donnée non validée par l'utilisateur n'est utilisée pour un calcul. Ensuite, la boucle « Analyse → Calcul → Réponse » peut se répéter autant de fois que l'utilisateur pose de nouvelles questions, dans le cadre d'une même conversation, sans repasser par les étapes d'import et d'extraction.

Conseil

L'export du rapport PDF (dernière étape du cycle) est optionnel et peut être déclenché à tout moment après la phase d'analyse, y compris après plusieurs échanges avec l'assistant conversationnel.

Résumé du chapitre

Le cycle complet d'une analyse GrapheinAI suit neuf étapes : import, prétraitement, extraction, validation, analyse, calcul, réponse, rapport — avec une pause de contrôle humain obligatoire après l'extraction.

La boucle question-réponse peut se répéter sans repasser par l'import, grâce à la mémoire conversationnelle.

9. Cas d'utilisation

Ce chapitre illustre, à travers cinq profils représentatifs mentionnés dans la documentation de GrapheinAI (README.md, section RecommendationEngine), la façon dont la plateforme s'adapte à des besoins métiers différents.

L'analyste financier

Le problème. Doit extraire rapidement les chiffres d'un graphique de résultats trimestriels reçu en image dans un rapport PDF, et vérifier des taux de croissance sans risquer d'erreur de calcul.

L'utilisation de GrapheinAI. Importe le graphique, valide les données extraites dans la grille HITL, puis pose des questions de type calcul (« quelle est la croissance du chiffre d'affaires entre T1 et T2 ? »), évaluées de façon déterministe par SafeCalculator.

Le résultat obtenu. Obtient des chiffres exacts et un rapport PDF prêt à être intégré à sa présentation, avec des recommandations adaptées au secteur Finance générées par le RecommendationEngine.

Le chercheur

Le problème. Doit extraire des séries de données à partir de graphiques scientifiques présents dans des publications, souvent sous forme d'images ou de PDF scannés, sans accès aux données brutes.

L'utilisation de GrapheinAI. Importe la figure, s'appuie sur l'extraction hybride OpenCV + Gemini pour obtenir les points de données, puis vérifie les valeurs extraites avant de lancer une analyse statistique (tendances, anomalies).

Le résultat obtenu. Récupère des données structurées exploitables pour ses propres analyses, avec une traçabilité complète (XAI) sur la façon dont chaque donnée a été obtenue.

L'étudiant

Le problème. N'a pas de compétences en développement ou en data science, mais doit comprendre et exploiter un graphique complexe pour un exposé ou un mémoire.

L'utilisation de GrapheinAI. Utilise l'interface simple d'import et pose des questions en langage naturel à l'assistant conversationnel, qui explique les tendances observées dans le graphique.

Le résultat obtenu. Comprend le contenu du graphique sans compétence technique préalable, et peut exporter une synthèse claire pour son travail académique.

Le responsable commercial

Le problème. Doit interpréter rapidement un graphique de ventes reçu par email pour préparer une réunion, sans avoir le temps de ressaisir les données dans un tableur.

L'utilisation de GrapheinAI. Importe le graphique, valide les données, puis interroge directement GrapheinAI sur les tendances et les comparaisons entre périodes ou régions.

Le résultat obtenu. Dispose en quelques minutes d'une synthèse narrative et de recommandations adaptées au secteur, prêtes à être présentées en réunion.

Le data analyst

Le problème. Doit intégrer des données provenant de graphiques hérités (legacy) dans ses propres analyses, et a besoin d'un export structuré plutôt que d'une simple lecture visuelle.

L'utilisation de GrapheinAI. Utilise le workflow complet : extraction, validation HITL, puis export Excel structuré multi-feuilles des données validées pour les réutiliser dans ses propres outils.

Le résultat obtenu. Gagne un temps considérable par rapport à une ressaisie manuelle, avec des données dont la fiabilité a été contrôlée par un score de confiance et une validation humaine.

Résumé du chapitre

Les cinq profils d'utilisateurs illustrent la polyvalence de GrapheinAI, de l'analyste financier au data analyst, en passant par des profils non techniques comme l'étudiant.

Dans chaque cas, le bénéfice combine gain de temps, fiabilité des données et adaptation du résultat au contexte métier.

10. Les points forts

Ce chapitre met en avant ce qui différencie GrapheinAI d'une approche classique — que ce soit une simple ressaisie manuelle, un OCR générique, ou un chatbot IA sans garde-fou. Chaque point s'appuie exclusivement sur les éléments documentés dans le projet.

- **Anti-hallucination déterministe :** contrairement à un assistant IA généraliste qui peut « inventer » un résultat numérique, GrapheinAI recalcule systématiquement chaque expression mathématique via un analyseur d'arbre syntaxique Python (SafeCalculator), garantissant une exactitude à 100 %.
- **Extraction hybride robuste :** la combinaison d'OpenCV (géométrie précise) et de Gemini Flash Vision (compréhension sémantique) est plus fiable qu'une approche reposant sur une seule de ces deux techniques.
- **Contrôle humain intégré (HITL) :** aucune donnée n'est analysée sans validation explicite de l'utilisateur, ce qui évite une confiance aveugle dans l'IA tout en conservant la rapidité de l'automatisation.
- **RAG local et sobre :** l'index vectoriel FAISS, exécuté localement, réduit selon la documentation jusqu'à 75 % les appels aux IA cloud énergivores, en traitant localement les calculs et recherches simples.
- **Transparence par conception (XAI) :** dix indicateurs de traçabilité documentent chaque réponse (données utilisées, formules évaluées, sources RAG, temps d'exécution, version du modèle), un niveau d'auditabilité rare pour ce type d'outil.
- **Sécurité multi-tenant de niveau entreprise :** l'isolation des données par Row Level Security PostgreSQL, combinée au RBAC et au PromptInjectionGuard, rend la plateforme directement utilisable par plusieurs clients sans développement de sécurité additionnel.
- **Sobriété numérique du frontend :** l'interface SPA en HTML5/Vanilla CSS pèse moins de 200 Ko, garantissant un chargement quasi instantané sans dépendance à des frameworks JavaScript lourds.
- **Bilinguisme natif :** la détection automatique de langue (français/anglais) s'applique à l'interface, aux réponses de l'IA et aux rapports PDF.
- **Exports professionnels prêts à l'emploi :** les rapports PDF vectoriels et fichiers Excel générés sont directement exploitables en contexte professionnel, sans reformatage manuel.

Résumé du chapitre

Le principal différenciateur de GrapheinAI est la combinaison de la puissance de l'IA générative avec la rigueur du calcul déterministe et du contrôle humain.

La sécurité, la sobriété numérique et la transparence (XAI) sont pensées comme des piliers du projet, pas comme des ajouts secondaires.

11. Limites

Les documents fournis (README.md et Architecture.md) ne comportent pas de section dédiée aux limitations du projet. Les points ci-dessous sont donc présentés avec honnêteté sur leur origine : il s'agit soit d'informations absentes des documents fournis, soit de limites structurelles que l'on peut raisonnablement déduire de l'architecture documentée.

11.1. Limitations déductibles de l'architecture documentée

- Dépendance à un service cloud externe : l'étape de raisonnement multimodal repose sur l'API Google Gemini Flash Vision. Une indisponibilité ou une latence de ce service impacte directement l'extraction et l'analyse complexe, malgré le mécanisme de retry/backoff (Tenacity) documenté.
- Analyse limitée aux graphiques statiques : le pipeline documenté traite des images et PDF de graphiques (histogrammes, courbes, camemberts, nuages de points, planches multi-graphiques). Les documents fournis ne mentionnent pas de prise en charge de tableaux de bord dynamiques, de vidéos ou de flux de données en temps réel.
- Formats de fichiers limités : seuls PNG, JPEG, WEBP et PDF (moins de 20 Mo) sont explicitement mentionnés comme formats acceptés.
- Deux langues supportées : le LanguageManager documenté couvre le français et l'anglais ; aucune autre langue n'est mentionnée.

11.2. Informations non documentées dans les fichiers fournis

- Aucune métrique quantitative de précision d'extraction (taux d'erreur, benchmark) n'est fournie au-delà des catégories qualitatives de confiance (VERY_HIGH, HIGH, MEDIUM, LOW).
- La suite de tests est mentionnée (182 tests, 28 fichiers), mais aucun taux de couverture de code n'est précisé.
- Les quotas d'API, limites de débit (rate limiting) ou plafonds d'usage ne sont pas détaillés dans README.md ou Architecture.md ; ils sont potentiellement couverts par ADMIN_GUIDE.md, un fichier référencé mais non fourni.
- Aucune option de déploiement on-premise (hors Docker/Render/Heroku/Streamlit Cloud) n'est mentionnée.

11.3. Évolutions envisagées

Les fichiers fournis ne présentent pas de feuille de route (roadmap) officielle. Aucune évolution future ne peut donc être affirmée comme prévue par l'équipe du projet. Les pistes suivantes sont de simples suggestions génériques, cohérentes avec les limites identifiées ci-dessus, et non des fonctionnalités annoncées :

- Élargir la prise en charge à d'autres langues au-delà du français et de l'anglais.
- Documenter ou introduire des métriques quantitatives de précision d'extraction, pour objectiver le score de confiance qualitatif existant.
- Explorer une option de secours locale pour le raisonnement multimodal, afin de réduire la dépendance à un service cloud unique.

Résumé du chapitre

Aucune limitation officielle n'est documentée dans les fichiers fournis ; ce chapitre distingue clairement les limites déductibles de l'architecture des informations simplement absentes.

La dépendance à l'API Gemini cloud est la limite structurelle la plus significative identifiable dans l'architecture documentée.

Les évolutions présentées sont des pistes génériques et non une feuille de route officielle du projet.

12. Glossaire

Ce glossaire explique simplement les termes techniques employés dans cette documentation.

Terme	Définition simple
SaaS (Software as a Service)	Un logiciel accessible directement en ligne, sans installation, généralement sur abonnement.
VLM (Vision-Language Model)	Un modèle d'IA capable de comprendre à la fois une image et du texte, et de raisonner sur les deux ensemble.
OCR (reconnaissance optique de caractères)	Une technologie qui détecte et lit le texte présent dans une image.
AST (Abstract Syntax Tree)	Une représentation structurée d'une expression (mathématique ou de code), qui permet de l'analyser et de l'évaluer sans exécuter du code arbitraire.
RAG (Retrieval-Augmented Generation)	Une technique qui enrichit la réponse d'une IA en allant d'abord chercher des informations pertinentes dans une base de connaissances existante.
FAISS	Une base de données vectorielle locale, développée par Meta, spécialisée dans la recherche des éléments les plus proches d'une requête.
Embedding	Une représentation numérique (un vecteur de nombres) qui capture le sens d'un texte, permettant de comparer des phrases entre elles.
XAI (Explainable AI / IA explicable)	Un ensemble de pratiques visant à rendre compréhensible et traçable le raisonnement d'une IA.
HITL (Human-in-the-Loop)	Un principe de conception où un humain valide ou corrige le résultat d'une IA avant qu'il ne soit utilisé plus loin dans un processus.
RLS (Row Level Security)	Un mécanisme de base de données qui filtre automatiquement les lignes qu'un utilisateur peut voir, selon des règles définies au niveau de la base elle-même.
RBAC (Role-Based Access Control)	Un système de droits d'accès fondé sur des rôles prédéfinis (par exemple administrateur, analyste, lecteur).
JWT (JSON Web Token)	Un jeton numérique signé, utilisé pour prouver l'identité d'un utilisateur authentifié lors de ses requêtes.
Multi-tenant	Une architecture où plusieurs clients (« locataires ») utilisent la même plateforme tout en ayant leurs données strictement isolées les unes des autres.
ASGI (Asynchronous Server Gateway Interface)	Une norme Python permettant à un serveur web de traiter plusieurs requêtes en parallèle de façon asynchrone.
Pydantic	Une bibliothèque Python qui valide automatiquement la structure et le type des données échangées par une application.
Gradient Boosting / XGBoost	Une famille d'algorithmes d'apprentissage automatique qui combine de nombreux petits modèles simples pour obtenir une prédiction précise.
Score de confiance	Une estimation, calculée automatiquement, du degré de fiabilité d'une donnée extraite (par exemple VERY_HIGH, HIGH, MEDIUM, LOW).
Magic bytes	Une suite d'octets caractéristique au tout début d'un fichier, permettant de vérifier son vrai type sans se fier à son extension.
Cache LRU (Least Recently Used)	Une mémoire tampon qui conserve les résultats les plus récemment utilisés et supprime automatiquement les plus anciens lorsque l'espace manque.
Webhook / Endpoint	Une adresse (URL) exposée par une application, à laquelle on peut envoyer une requête pour déclencher une action ou obtenir une information.

Résumé du chapitre

Terme	Définition simple
Ce glossaire couvre les principaux acronymes et notions techniques mobilisés par l'architecture de GrapheinAI, expliqués sans jargon superflu.	

13. FAQ

Cette FAQ répond aux questions les plus probables qu'un utilisateur débutant, ou un examinateur technique, pourrait se poser sur GrapheinAI.

Q. GrapheinAI remplace-t-il un analyste humain ?

R. Non. La conception même du produit repose sur le principe Human-in-the-Loop : l'IA propose une extraction et une analyse, mais l'utilisateur valide ou corrige les données avant toute exploitation. GrapheinAI est un outil d'assistance, pas un substitut à la décision humaine.

Q. Mes données sont-elles utilisées pour entraîner des modèles publics ?

R. D'après la documentation, non : aucune donnée client ou image de graphique transmise n'est réutilisée pour le ré-entraînement de modèles d'IA publics, conformément à l'engagement RGPD affiché par le projet.

Q. Quels formats de fichiers puis-je importer ?

R. Les formats explicitement documentés sont PNG, JPEG, WEBP et PDF, avec une taille maximale de 20 Mo, vérifiée via le contrôle des « magic bytes ».

Q. Comment GrapheinAI évite-t-il les erreurs de calcul de l'IA ?

R. Toute expression mathématique proposée par le modèle d'IA générative est recalculée par le module SafeCalculator, un évaluateur déterministe basé sur l'analyse d'arbre syntaxique Python, qui n'autorise que des opérations arithmétiques et statistiques précises.

Q. Ai-je besoin de compétences techniques pour utiliser GrapheinAI ?

R. Non. L'interface Web SPA et l'application Streamlit sont conçues pour un usage sans connaissance en développement, en intelligence artificielle ou en data science.

Q. Puis-je collaborer avec mon équipe sur une même analyse ?

R. Oui, via les espaces de travail (workspaces), qui permettent de partager des sessions d'analyse entre plusieurs membres, avec des rôles différenciés (administrateur, analyste, lecteur).

Q. Quelles langues sont prises en charge ?

R. Le français et l'anglais, avec détection automatique de la langue via le LanguageManager, appliquée à l'interface, aux réponses de l'IA et aux rapports générés.

Q. Comment exporter les résultats d'une analyse ?

R. Deux formats d'export sont disponibles : un rapport PDF vectoriel professionnel (via ReportLab) et un classeur Excel structuré multi-feuilles (via OpenPyXL et XlsxWriter).

Q. GrapheinAI fonctionne-t-il sans connexion internet ?

R. Ce point n'est pas précisé explicitement dans les documents fournis. On peut toutefois noter que l'étape de recherche vectorielle (FAISS) est locale, tandis que l'étape de raisonnement multimodal dépend de l'API cloud Google Gemini, ce qui suggère qu'une connexion réseau reste nécessaire pour les fonctionnalités d'extraction et d'analyse avancée.

Q. Comment est calculé le score de confiance d'une extraction ?

R. Le ValidationAgent le calcule en croisant trois signaux : l'alignement OCR, le chevauchement géométrique des zones détectées, et la plausibilité numérique des valeurs, pour aboutir à une catégorie qualitative (VERY_HIGH, HIGH, MEDIUM, LOW).

Q. Comment mes données sont-elles protégées vis-à-vis des autres clients de la plateforme ?

R. Par les politiques Row Level Security (RLS) de PostgreSQL au niveau de Supabase, qui garantissent qu'un utilisateur ne peut accéder qu'à ses propres données ou à celles de son espace de travail, quel que soit le code applicatif exécuté au-dessus.

Q. Quel est le niveau de qualité et de test du projet ?

R. La documentation mentionne une suite de 28 fichiers de tests automatisés, totalisant plus de 180 assertions (pytest), ainsi que l'usage de Ruff (linting) et Black (formatage) pour la qualité du code. Aucun taux de couverture de code n'est précisé.

Résumé du chapitre

Cette FAQ couvre les interrogations les plus courantes sur la confiance, la sécurité, les formats et les usages de GrapheinAI, en s'appuyant exclusivement sur les informations documentées.

14. Conclusion

Cette documentation a présenté GrapheinAI dans son intégralité : le problème métier qui l'a motivé, son fonctionnement du point de vue de l'utilisateur, ses fonctionnalités, son architecture technique, ses technologies et son pipeline d'intelligence artificielle.

14.1. Les objectifs

GrapheinAI répond à un besoin concret et répandu : transformer des graphiques figés en images en données exploitables, fiables et interprétées, sans les erreurs et la lenteur d'une ressaisie manuelle.

14.2. Les innovations

Le projet se distingue par la combinaison rare de plusieurs approches habituellement dissociées : la vision par ordinateur déterministe, l'IA générative multimodale, le calcul mathématique garanti sans hallucination (SafeCalculator AST), la recherche vectorielle locale et sobre (FAISS), et l'explicabilité systématique (XAI) de chaque résultat produit.

14.3. Les technologies

GrapheinAI mobilise un ensemble cohérent de technologies modernes : FastAPI et Supabase pour le socle applicatif et la sécurité multi-tenant, OpenCV et Gemini Flash Vision pour l'extraction hybride, XGBoost et FAISS pour le routage intelligent des requêtes, et ReportLab pour la production de livrables professionnels.

14.4. Les compétences démontrées

La conception d'une plateforme comme GrapheinAI mobilise des compétences transverses : ingénierie logicielle backend (API REST, architecture hexagonale), intelligence artificielle appliquée (vision par ordinateur, modèles de langage multimodaux, RAG, machine learning supervisé), sécurité et conformité (RLS, RBAC, RGPD), ainsi qu'une réflexion produit centrée sur la fiabilité et la sobriété numérique.

14.5. Les perspectives

Comme indiqué au chapitre 11, les documents fournis ne présentent pas de feuille de route officielle. Les axes de progression les plus naturels, déduits des limites identifiées, concernent l'élargissement des langues supportées, l'objectivation quantitative des scores de confiance, et une réduction accrue de la dépendance à un unique fournisseur d'IA cloud.

Résumé du chapitre

GrapheinAI transforme un problème métier concret — l'exploitation fiable de données graphiques figées — en une plateforme SaaS complète, sécurisée et transparente.

Sa spécificité tient à l'articulation entre IA générative et garde-fous déterministes, plutôt qu'à l'IA générative seule.

Cette documentation constitue une base solide pour un onboarding développeur, une soutenance ou une présentation à un recruteur technique.